Vivekanand Education Society's Institute Of Technology (VESIT)
Department of Information Technology
Course: Blockchain

Mini Project
Cookbook

Decentralized Personal Health Record (PHR) dApp

Storing Personal Health Records using Ethereum Blockchain and the Interplanetary File System (IPFS)

Roll No.	Name
47	Nikhil Kale
56	Aayush Patil
57	Yogita Patil
61	Varun Rahatgaonkar

Group No.: _____
Date:
Faculty In-Charge: Kajal Jewani
Repository: https://github.com/YogitaPatil12095/MiniProject-Blockchain-
Base paper: *Sentausa & Hareva, "Decentralize Application for Storing Personal Health Record using Ethereum Blockchain and Interplanetary File System," ICTIIA 2022 (DOI 10.1109/ICTIIA54654.2022.9935996)*

1. Overview

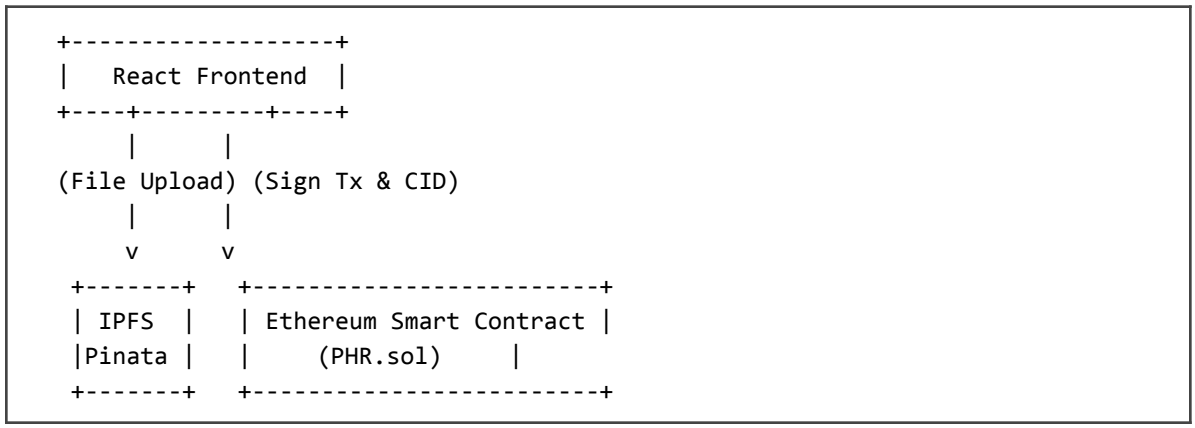
This dApp lets a patient own their medical record and decide which wallet addresses may view or add to it. Medical files are uploaded to IPFS via Pinata to keep on-chain storage cheap; only the tamper-evident IPFS Content Identifier (CID), the creator's address, and a timestamp are written to a single Ethereum smart contract (PHR.sol). The project reproduces the system, test plan, and gas-cost benchmarks published in the base IEEE paper listed on the title page, and declares where this implementation deviates from it.

Two roles use the system:

- Patient — owns a record; can register, view their own records, view their access list, grant access, and revoke access.
- Doctor — any registered user who has been granted access; can view a patient's records (needs Viewer access) or create a record for a patient (needs Creator access).

2. Architecture Overview

Medical files never touch the blockchain directly. The flow is:



Reads (viewing records, access lists) are free calls made through a signer-connected contract instance. Writes (register, grant/revoke access, create a record) require a MetaMask confirmation and cost gas because they change on-chain state.

3. Implemented Features

ID	Feature	Description	Gas?
FR-1	Login & Register	Connect MetaMask; first-time callers register via setUserData.	Yes (first time)
FR-2	View Health Records	Patient views all their own records and opens files via IPFS links.	No (free query)
FR-3	View Access List	Patient inspects their authorized Viewers and Creators.	No (free query)

FR-4	Grant Access	Patient grants an address Viewer (v), Creator (c), or Master (m) access.	Yes
FR-5	View Patient Records	Doctors view records for patients who granted them Viewer access.	No (free query)
FR-6	Create Patient Record	Doctors upload a file to IPFS and anchor the CID via createEHR.	Yes
FR-7	Revoke Access	Patient removes viewing or creating permission from an address (added feature, not in the base paper).	Yes

4. Core Stack

Layer	**Tooling**
Smart contract	Solidity 0.8.24, single contract PHR.sol
Contract framework	Hardhat 2.22.x, @nomicfoundation/hardhat-toolbox, hardhat-gas-reporter
Frontend	React 18, Vite 5, react-router-dom
Chain interaction	ethers v6
Wallet	MetaMask browser extension
File storage	IPFS via Pinata (CID only stored on-chain)
Networks	Hardhat local (chainId 31337), Ethereum Sepolia testnet (chainId 11155111)
Icons / UI	lucide-react

5. Prerequisites

6. Install Dependencies

```
# 1. Clone repository
git clone https://github.com/YogitaPatil12095/MiniProject-Blockchain-.git
cd MiniProject-Blockchain-

# 2. Install root (Hardhat / contract) dependencies
npm install

# 3. Install frontend dependencies
cd frontend && npm install && cd ..
```

7. Configuration

7.1 Root environment (.env or .env.local)

Used by hardhat.config.js for Sepolia deployment and the gas reporter:

```
SEPOLIA_RPC_URL=https://sepolia.infura.io/v3/<project-id>
DEPLOYER_PRIVATE_KEY=0x<your-private-key>
COINMARKETCAP_API_KEY=<optional, for USD gas pricing in the report>
```

7.2 Frontend environment

Pinata credentials used by frontend/src/lib/ipfs.js for file uploads (add to frontend/.env):

```
VITE_PINATA_JWT=<your-pinata-jwt>
VITE_PINATA_GATEWAY=https://gateway.pinata.cloud
```

8. Running Tests & Gas Reporter

```
npx hardhat test
```

This compiles PHR.sol, runs the full test suite (see Section 12), and prints a gas-usage table generated by hardhat-gas-reporter alongside the pass/fail results.

9. Running the Local dApp

Terminal 1 — start the local Hardhat blockchain

```
npx hardhat node
```

Note the 20 pre-funded test accounts (10,000 ETH each) and their private keys printed to the console — you will import two or three of these into MetaMask.

Terminal 2 — deploy the contract

```
npx hardhat run scripts/deploy.js --network localhost
```

This deploys PHR.sol and automatically writes the address, ABI, and chain ID to frontend/src/contract.json, which the frontend reads at startup.

Terminal 3 — start the frontend

```
cd frontend
npm run dev
# open http://localhost:3000
```

10. Configuring MetaMask for Local Testing

1. Open MetaMask → Network Selector (top left) → Add a network manually.

- Network Name: Hardhat Local
- New RPC URL: http://127.0.0.1:8545
- Chain ID: 31337

- Currency Symbol: ETH

2. Import test accounts: Account selector → Add account or hardware wallet → Import account.

- Copy Private Key #0 from the hardhat node console for the Patient account.
- Copy Private Key #1 for the Doctor account.
- Each imported account should show a balance of 10,000 ETH.

11. End-to-End Demo Workflow

1. Patient Registration — connect Account 1 (Patient); fill in Full Name, Gender, Address, Phone, Birthday and submit setUserData.

2. Doctor Registration — switch MetaMask to Account 2 (Doctor); fill in the registration form and submit.

3. Grant Master Access — switch back to Account 1; go to Grant Access, paste Account 2's address, select Master, confirm in MetaMask.

4. Doctor Uploads EHR — switch to Account 2; go to Doctor Portal → Create Patient Record, paste the patient address, pick a sample file, click Upload & Save (file goes to IPFS, CID is anchored on-chain via createEHR).

5. Patient Inspects Records — switch to Account 1; go to My Records; click Open on IPFS to view the uploaded document.

6. Access Control Check — switch to Account 3 (unregistered / non-granted); attempt to view patient records; the call is rejected with a friendly error ("The patient has not given you permission to view their records").

12. Testing Method

The test suite is written against Hardhat's Waffle/Chai matchers and mirrors the base paper's five test sections, plus an extra section for features added in this implementation (revoke access, duplicate-grant checks, edge cases):

	Section	**What it checks**
1	Creating New User	Patient, Doctor, User1 (creator role), and User2 (viewer role) all register without errors on first-time login.
2	Granting Access	Each of the four dummy users ends up with the correct access role (viewer / creator / master) after grantAccess.
3	Viewing Access List	Each user can read back its own viewer list and creator list via getMyAccessList.
4	Viewing EHR	Only an address with granted viewer access can call viewEHR successfully; others are rejected.
5	Creating EHR	Only an address with granted creator access can call createEHR successfully; others are rejected.

6	Extra Tests	revokeAccess, duplicate-grant/duplicate-registration guards, and other edge cases not covered by the base paper.

Run with:

```
npx hardhat test
```

13. Gas Cost Comparison

Reference gas figures from the base paper (Fig. 22) compared with this implementation's measured gas usage. Differences come from the added canView/canCreate mappings, events, and the newer Solidity 0.8.24 compiler:

Method	**Min Gas**	**Max Gas**	**Our Avg Gas**	**Paper Avg (Fig 22)**
setUserData	250,740	250,776	250,758	206,029
grantAccess	79,387	146,850	102,631	75,142
createEHR	143,750	143,930	143,894	203,904
revokeAccess	39,537	60,953	47,446	N/A (added feature)
Deployment	—	—	1,740,814 (2.9% block limit)	2,766,773

14. Smart Contract Reference (PHR.sol)

Single contract, two structs (User, EHR), and the following external functions:

Function	**Caller**	**Effect**
setUserData(name, gender, address, phone, birthday)	Anyone (once)	Registers the caller; caller becomes a viewer of their own record.
grantAccess(target, role)	Registered patient	Grants target Viewer ("v"), Creator ("c"), or Master ("m") access to the caller's records.
revokeAccess(target, role)	Registered patient	Removes previously granted Viewer, Creator, or Master access from target.
createEHR(patient, creatorName, ipfsLocation)	Address with Creator access	Appends a new EHR entry (with block.timestamp) to the patient's record.
viewEHR(patient)	Address with Viewer access	Returns the patient's full EHR array (free call).
getUserData(user)	Anyone	Returns a registered user's demographic fields (free call).
getMyAccessList()	Registered caller	Returns the caller's own viewerList and creatorList (free call).

isUserRegistered / isGrantedToView / isGrantedToCreate	Anyone	Boolean lookups used by the frontend to gate UI actions (free calls).

15. Declared Deviations from the Base Paper

Paper	This project	Reason
Client-supplied creation date	block.timestamp	Cannot be forged by the caller.
Array scans for access checks	canView / canCreate mappings for checks; arrays kept for listing	O(1) lookups, cheaper gas.
No revoke feature	revokeAccess added	Gives the patient real control over previously granted access.
No contract events	Events emitted on every state change	Simplifies frontend updates and auditing.
Repeatable setUserData	One-time registration per address	Prevents an address from overwriting its profile.
Rinkeby testnet	Hardhat local network + Sepolia	Rinkeby has been deprecated and shut down.

16. Known Limitations

- Everything on-chain is publicly readable regardless of Solidity's public/private keyword — contract-level access control only restricts contract calls, not raw storage reads.
- Files on public IPFS are accessible to anyone who has the CID; there is no client-side encryption in this version.
- Any wallet address can self-register as a user — there is no centralized doctor/medical-board verification.
- The Pinata JWT is exposed in the browser in this demo setup.
- CIDs are immutable, so a record cannot be deleted — only access to the pointer can be revoked.

Future Work

- Client-side encryption with per-viewer key sharing.
- Admin-verified doctor onboarding.
- A backend proxy for IPFS uploads, so the Pinata key never reaches the browser.
- Soft-delete / record-revocation flags.

17. Repository Map

```
MiniProject-Blockchain-/
  contracts/
    PHR.sol          Main smart contract
  test/
    PHR.test.js        Hardhat/Waffle test suite (6 sections)
  scripts/
    deploy.js        Deploys PHR.sol, writes frontend/src/contract.json
  hardhat.config.js  Solidity version, networks, gas reporter
  frontend/
       src/
      pages/           LoginRegisterPage, PatientDashboardPage,
                       DoctorDashboardPage, PreviewPage
      components/ui/   Presentational components (Button, Input,
                       RecordCard, RoleBadge, GasBadge, ...)
      context/         WalletContext.jsx
      hooks/         usePHR.js
       lib/          contract.js, ipfs.js, constants.js
      contract.json  Auto-generated — never hand-edit
  PRD.md, DESIGN.md, MILESTONES.md, ROLES.md   Project documentation
```

18. Troubleshooting

Symptom	Fix
MetaMask shows “Wrong network”	Confirm the Hardhat Local network is selected and its Chain ID is set to 31337.
Frontend can't find the contract / calls revert unexpectedly	Re-run the deploy script (Section 9, Terminal 2) so frontend/src/contract.json is regenerated, then refresh the page.
“Already registered” on setUserData	That address has already called setUserData once; registration is one-time per address by design.
“You are not granted as a creator/viewer”	The patient has not run Grant Access for that address yet, or granted the wrong role.
File upload to IPFS fails	Check VITE_PINATA_JWT is set in frontend/.env and the Pinata account/plan has remaining upload quota.
npx hardhat test fails to compile	Confirm Node.js is v20 or v22 and that npm install was run at the repository root.
Sepolia deployment fails	Confirm SEPOLIA_RPC_URL and DEPLOYER_PRIVATE_KEY are set in .env and the deployer address holds Sepolia test ETH.

19. References

1. R. Sentausa and D. H. Hareva, “Decentralize Application for Storing Personal Health Record using Ethereum Blockchain and Interplanetary File System,” 2022 1st International Conference on Technology Innovation and Its Applications (ICTIIA), 2022. DOI: 10.1109/ICTIIA54654.2022.9935996.

2. Project repository: https://github.com/YogitaPatil12095/MiniProject-Blockchain-

3. Hardhat documentation: https://hardhat.org/

4. Ethers v6 documentation: https://docs.ethers.org/v6/

5. Pinata (IPFS pinning service): https://www.pinata.cloud/